

Course	CSC150
Name	Program 3 – Converting units
Due Date	November 8, 2019 at 11:59pm MST
Repository	csc150f19programs
Solution Name	prog3
Project Files	prog3.cpp
Build Type	X64

SETUP:

Create a solution named prog3 in your program repository (csc150f19programs). Add a single source file named prog3.cpp to this project. Make sure you have an x64 bit program.

PURPOSE:

The purpose of this assignment is focused on giving you practice with functions.

PROBLEM:

One of our instructors has recently moved to the USA and an unexpected challenge in adaptation is due to differences in unit systems (metric versus imperial). So, we are asking you to help him to convert data. There is also the slight chance that the instructors are simply using this as an excuse for you to practice functions in C++. 😊

The description below mentions the names of functions you need to create, but you may need to choose the appropriate function type and any necessary parameters for each of your functions.

TIER 1: MAIN MENU

First, you will output the title “Unit Conversion Tool”, followed by an empty line. Then, you must call a **void** function named **show_main_menu(...)**, to display the main menu with general options. If the user selects an invalid option, you should display “Invalid option”, and show the menu again. If the user selects “X”, your program should output “Goodbye!” and stop execution. Note that all options are CASE-INSENSITIVE.

There are 20 dashes in the line under “Main Menu:”

```
Unit Conversion Tool
```

```
Main Menu:
```

```
-----
```

```
(T) Temperature
```

```
(D) Distance
```

```
(V) Volume
```

```
(X) Exit
```

```
Select the type of conversion:
```

TIER 2: TEMPERATURE

If the user selected option “T” in the main menu, you will show a sub-menu as follows:

Temperature conversion:

(F) From Celsius to Fahrenheit

(C) From Fahrenheit to Celsius

(X) Back to main menu

As in the main menu, if user types an invalid option, you will show an error message “Invalid Option” and display the menu again.

Based on the user’s input you will output “Enter the temperature in Celsius: ” or “Enter the temperature in Fahrenheit: ”, and wait for the user to enter a number.

You will calculate the conversion, and display the results:

“XX degrees Celsius is equivalent to YY degrees Fahrenheit” or “XX degrees Fahrenheit is equivalent to YY degrees Celsius”.

Then you will return to the main menu, displaying all of its options.

To calculate the conversion, you will need to create two **valued** functions: **celsius_to_fahrenheit(...)** and **fahrenheit_to_celsius(...)**. The formulas to convert between units are as follows:

$$F = \frac{C \times 9}{5} + 32 \text{ or } C = (F - 32) \times \frac{5}{9}$$

TIER 3: DISTANCE

If the user selected option “T” in the main menu, you will show a sub-menu as follows:

Distance conversion:

(M) Miles to Metric

(I) Kilometers to Imperial

(X) Back to main menu

As previously, if user types an invalid option, you will show an error message “Invalid Option” and display the menu again.

Based on the user’s input you will output “Enter the distance in Miles: ” or “Enter the distance in Kilometers: ”, and wait for the user to enter a number.

You will calculate the conversion, and display the results:

“XX mi is equivalent to YY km or ZZ m” or “XX Km is equivalent to YY mi or ZZ ft”.

To convert distances, you will use the **void** functions **miles_to_metric(...)** and **km_to_imperial(...)** and know that $1mi = 5280ft = 1609.34m$ and $1km = 1,000m$.

TIER 4: VOLUME

If the user selected option “V” in the main menu, you will ask for the volume to convert (a real number), then the source and destination units, as the following example (**red bold** for user input).

Volume conversion:

Enter the volume to convert: **1**

Select unit to convert from (L for liter, G for gallons, or Q for Quarters): **x**

Invalid option!

Select unit to convert from (L for liter, G for gallons, or Q for Quarters): **G**

Select unit to convert to (L for liter, G for gallons, or Q for Quarters): **L**

You have approximately to 4.546 l

You will need to write one valued (double) function named **convert_volume(...)** for the calculation. You will define the number and types of parameters of this function, but the function should return the converted volume as a double and, as a reference parameter, return a string with the symbol of the converted volume, according to the following table:

Unit	Symbol	Conversion
Liter	l	
Gallon	gal	1 gal = 4.546 l
Quarter	Qt	1 qt = .25 gal

ADDITIONAL REQUIREMENTS

- Compiler warnings will now be a -3 points per warning
- You may not have a blank line between every line of code. Doing so will be -5 points.
- Documentation now has **even** HIGHER additional deductions.
- Make sure that you DO NOT USE global variables. Global constants are okay, and even encouraged (and some may even be provided to you).
- At a minimum, your code needs to have the following functions (all described above), but you will need to define any necessary parameter to your functions. You can create any other functions you may want to organize your code.
 - void show_main_menu(...): This function outputs main menu.
 - celsius_to_fahrenheit(...): This function takes in the temperature value in celsius, converts and returns the temperature value in fahrenheit.
 - fahrenheit_to_celsius(...): This function takes in the temperature value in fahrenheit, converts and returns the temperature value in celsius.
 - void km_to_imperial(...): This function takes in the distance value in kilometers, converts and returns the distance value in both miles and feet, as two separate variables.
 - void miles_to_metric(...): This function takes in the distance value in miles, converts and returns the distance value in both kilometers and meters, as two separate variables.
 - double convert_volume(...): See description in Tier 4.